SQL Code

```sql
create database RetailDB_3;
use RetailDB_3;

CREATE TABLE Customers (
 customer_id INT AUTO_INCREMENT PRIMARY KEY,
 name VARCHAR(100),
 email VARCHAR(100),
 city VARCHAR(50),
 signup_date DATE
);

CREATE TABLE Suppliers (
 supplier_id INT AUTO_INCREMENT PRIMARY KEY,
 supplier_name VARCHAR(100),
 contact_email VARCHAR(100),
 city VARCHAR(50)
);

CREATE TABLE Shippers (
 shipper_id INT AUTO_INCREMENT PRIMARY KEY,
 shipper_name VARCHAR(100),
 contact VARCHAR(100)
);

CREATE TABLE Payment_Methods (
 payment_id INT AUTO_INCREMENT PRIMARY KEY,
 payment_type VARCHAR(50) UNIQUE
);

CREATE TABLE Products (
 product_id INT AUTO_INCREMENT PRIMARY KEY,
 product_name VARCHAR(100),
 category VARCHAR(50),
 price DECIMAL(10,2),
 stock_qty INT,
 supplier_id INT,
 FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id)
);

CREATE TABLE Orders (
 order_id INT AUTO_INCREMENT PRIMARY KEY,
 customer_id INT,
 order_date DATE,
 payment_id INT,
 shipper_id INT,
 FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
 FOREIGN KEY (payment_id) REFERENCES Payment_Methods(payment_id),
 FOREIGN KEY (shipper_id) REFERENCES Shippers(shipper_id)
);

```

```sql
CREATE TABLE Order_Items (
 order_item_id INT AUTO_INCREMENT PRIMARY KEY,
 order_id INT,
 product_id INT,
 quantity INT,
 price_each DECIMAL(10,2),
 FOREIGN KEY (order_id) REFERENCES Orders(order_id),
 FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

insert into Customers(customer_id,name,email,city,signup_date)values
(1, "Rohit Kumar", "rohit.kumar@gmail.com", "Delhi", "2023-01-12"),
(2, "Sneha Sharma", "sneha.sharma@yahoo.com", "Mumbai", "2023-02-05"),
(3, "Amit Patel", "amit.patel@gmail.com", "Ahmedabad", "2023-02-18"),
(4, "Priya Reddy", "priya.reddy@gmail.com", "Hyderabad", "2023-03-02"),
(5, "Karan Singh", "karan.singh@outlook.com", "Chennai", "2023-03-15"),
(6, "Neha Verma", "neha.verma@gmail.com", "Pune", "2023-04-01"),
(7, "Arjun Mehta", "arjun.mehta@gmail.com", "Bengaluru", "2023-04-20"),
(8, "Ritika Gupta", "ritika.gupta@yahoo.com", "Kolkata", "2023-05-12"),
(9, "Vikram Joshi", "vikram.joshi@gmail.com", "Lucknow", "2023-05-25"),
(10, "Ananya Das", "ananya.das@gmail.com", "Bhubaneswar", "2023-06-08"),
(11, "Suresh Iyer", "suresh.iyer@gmail.com", "Chennai", "2023-06-20"),
(12, "Megha Kapoor", "megha.kapoor@yahoo.com", "Jaipur", "2023-07-03"),
(13, "Ravi Shankar", "ravi.shankar@gmail.com", "Delhi", "2023-07-15"),
(14, "Tanya Mishra", "tanya.mishra@gmail.com", "Noida", "2023-08-01"),
(15, "Aditya Jain", "aditya.jain@gmail.com", "Indore", "2023-08-14");
insert into Suppliers(supplier_id,supplier_name,contact_email,city)values
(1, "Reliance Digital", "support@reliance.com", "Mumbai"),
(2, "Croma Electronics", "info@croma.com", "Delhi"),
(3, "Flipkart Seller Hub", "sellers@flipkart.com", "Bengaluru"),
(4, "Amazon India Vendor", "vendor@amazon.in", "Hyderabad"),
(5, "Tata Cliq", "support@tatacliq.com", "Pune"),
(6, "Vijay Sales", "sales@vijaysales.com", "Chennai"),
(7, "Lifestyle Stores", "info@lifestylestores.com", "Kolkata"),
(8, "Metro Electronics", "metro@electronics.com", "Ahmedabad"),
(9, "Big Bazaar", "help@bigbazaar.com", "Indore"),
(10, "D-Mart Suppliers", "supply@dmart.in", "Nagpur"),
(11, "Shoppers Stop", "contact@shoppersstop.com", "Jaipur"),
(12, "Snapdeal Vendors", "vendor@snapdeal.com", "Noida");
insert into Shippers(shipper_id,shipper_name,contact)values
(1, "BlueDart Express", "support@bluedart.com"),
(2, "DTDC Courier", "help@dtdc.com"),
(3, "Delhivery Logistics", "contact@delhivery.com"),
(4, "Ekart Logistics", "care@ekart.com"),
(5, "Gati Express", "service@gati.com"),
(6, "India Post Speed", "info@indiapost.gov.in"),
(7, "Shadowfax Delivery", "support@shadowfax.in");
insert into Payment_Methods(payment_id,payment_type)values
(1, "UPI"),
(2, "Credit Card"),
(3, "Debit Card"),
(4, "Net Banking"),
(5, "Cash"),
(6, "Wallet"),
```

```sql
(7, "EMI");
insert into Products(product_id,product_name,category,price,stock_qty,supplier_id)values
(1, "iPhone 15", "Electronics", 79999, 50, 1),
(2, "Samsung Galaxy S24", "Electronics", 74999, 40, 2),
(3, "Noise Smartwatch", "Wearables", 4999, 100, 3),
(4, "Boat Earbuds", "Wearables", 2499, 200, 4),
(5, "Kurta Set", "Fashion", 1999, 150, 7),
(6, "Running Shoes", "Fashion", 2999, 120, 7),
(7, "Prestige Mixer Grinder", "Home Appliances", 6499, 80, 5),
(8, "Sony Bravia 55” TV", "Electronics", 59999, 30, 6),
(9, "Lenovo Laptop", "Electronics", 55999, 45, 2),
(10, "Philips Trimmer", "Personal Care", 1499, 90, 8);
insert into Orders(order_id,customer_id,order_date,payment_id,shipper_id)values
(1, 1, "2024-01-05", 1, 3),
(2, 2, "2024-01-06", 2, 5),
(3, 3, "2024-01-07", 4, 2),
(4, 4, "2024-01-08", 3, 1),
(5, 5, "2024-01-08", 5, 7),
(6, 6, "2024-01-09", 7, 4),
(7, 7, "2024-01-09", 6, 6),
(8, 8, "2024-01-10", 2, 3),
(9, 9, "2024-01-11", 1, 5),
(10, 10, "2024-01-11", 3, 2),
(11, 11, "2024-01-12", 4, 1),
(12, 12, "2024-01-13", 5, 6),
(13, 13, "2024-01-14", 6, 4),
(14, 14, "2024-01-14", 7, 7),
(15, 15, "2024-01-15", 1, 2),
(16, 3, "2024-01-15", 2, 5),
(17, 7, "2024-01-16", 4, 6),
(18, 10, "2024-01-17", 5, 3),
(19, 2, "2024-01-17", 6, 1),
(20, 6, "2024-01-18", 3, 7),
(21, 8, "2024-01-19", 1, 4),
(22, 1, "2024-01-19", 7, 2),
(23, 4, "2024-01-20", 2, 6),
(24, 11, "2024-01-21", 5, 5),
(25, 9, "2024-01-21", 3, 7),
(26, 12, "2024-01-22", 4, 1),
(27, 14, "2024-01-23", 6, 4),
(28, 13, "2024-01-23", 2, 3),
(29, 5, "2024-01-24", 1, 6),
(30, 15, "2024-01-25", 7, 7),
(31, 8, "2024-01-25", 6, 2),
(32, 7, "2024-01-26", 3, 5),
(33, 2, "2024-01-27", 4, 1),
(34, 9, "2024-01-27", 2, 4),
(35, 10, "2024-01-28", 1, 6),
(36, 11, "2024-01-29", 5, 7),
(37, 14, "2024-01-30", 7, 3),
(38, 6, "2024-01-30", 3, 2),
(39, 3, "2024-01-31", 6, 5),
(40, 12, "2024-02-01", 2, 1),
```

```sql
(41, 1, "2024-02-01", 4, 6),
(42, 13, "2024-02-02", 5, 7),
(43, 15, "2024-02-02", 7, 4),
(44, 4, "2024-02-03", 1, 2),
(45, 5, "2024-02-04", 3, 6),
(46, 2, "2024-02-05", 6, 7),
(47, 7, "2024-02-05", 5, 1),
(48, 9, "2024-02-06", 2, 3),
(49, 8, "2024-02-07", 1, 4),
(50, 10, "2024-02-07", 7, 2),
(51, 11, "2024-02-08", 6, 5),
(52, 6, "2024-02-09", 3, 7),
(53, 14, "2024-02-09", 2, 6),
(54, 12, "2024-02-10", 5, 1),
(55, 13, "2024-02-10", 7, 4),
(56, 1, "2024-02-11", 4, 3),
(57, 15, "2024-02-12", 6, 2),
(58, 5, "2024-02-12", 3, 7),
(59, 4, "2024-02-13", 2, 5),
(60, 2, "2024-02-14", 1, 6),
(61, 3, "2024-02-14", 7, 1),
(62, 7, "2024-02-15", 6, 4),
(63, 8, "2024-02-15", 5, 2),
(64, 9, "2024-02-16", 4, 7),
(65, 10, "2024-02-16", 2, 6),
(66, 11, "2024-02-17", 3, 1),
(67, 12, "2024-02-18", 6, 5),
(68, 13, "2024-02-18", 1, 4),
(69, 14, "2024-02-19", 7, 3),
(70, 15, "2024-02-20", 5, 2),
(71, 1, "2024-02-21", 2, 7),
(72, 2, "2024-02-21", 3, 6),
(73, 3, "2024-02-22", 4, 1),
(74, 5, "2024-02-23", 6, 5),
(75, 7, "2024-02-23", 7, 4);
insert into Order_Items(order_item_id,order_id,product_id,quantity,price_each)values
(1, 1, 1, 2, 15000),
(2, 2, 3, 1, 12000),
(3, 3, 5, 3, 2500),
(4, 4, 2, 1, 22000),
(5, 5, 7, 2, 1800),
(6, 6, 4, 1, 8000),
(7, 7, 6, 4, 1500),
(8, 8, 8, 2, 3500),
(9, 9, 10, 1, 7000),
(10, 10, 9, 3, 1200),
(11, 11, 2, 1, 22000),
(12, 12, 1, 2, 15000),
(13, 13, 4, 3, 8000),
(14, 14, 7, 1, 1800),
(15, 15, 5, 2, 2500),
(16, 16, 3, 2, 12000),
(17, 17, 6, 3, 1500),
(18, 18, 9, 4, 1200),
```

```
(19, 19, 8, 1, 3500),
(20, 20, 10, 2, 7000),
(21, 21, 1, 1, 15000),
(22, 22, 2, 3, 22000),
(23, 23, 5, 2, 2500),
(24, 24, 7, 1, 1800),
(25, 25, 6, 2, 1500),
(26, 26, 4, 1, 8000),
(27, 27, 3, 1, 12000),
(28, 28, 8, 2, 3500),
(29, 29, 9, 1, 1200),
(30, 30, 10, 2, 7000),
(31, 31, 1, 3, 15000),
(32, 32, 2, 1, 22000),
(33, 33, 7, 2, 1800),
(34, 34, 4, 1, 8000),
(35, 35, 6, 3, 1500),
(36, 36, 5, 2, 2500),
(37, 37, 3, 1, 12000),
(38, 38, 9, 2, 1200),
(39, 39, 8, 1, 3500),
(40, 40, 2, 2, 22000),
(41, 41, 1, 2, 15000),
(42, 42, 4, 1, 8000),
(43, 43, 6, 3, 1500),
(44, 44, 7, 2, 1800),
(45, 45, 10, 2, 7000),
(46, 46, 3, 1, 12000),
(47, 47, 5, 2, 2500),
(48, 48, 9, 3, 1200),
(49, 49, 8, 2, 3500),
(50, 50, 1, 1, 15000),
(51, 51, 4, 2, 8000),
(52, 52, 2, 1, 22000),
(53, 53, 7, 3, 1800),
(54, 54, 6, 2, 1500),
(55, 55, 5, 1, 2500),
(56, 56, 10, 2, 7000),
(57, 57, 8, 1, 3500),
(58, 58, 9, 2, 1200),
(59, 59, 3, 1, 12000),
(60, 60, 2, 2, 22000),
(61, 61, 1, 2, 15000),
(62, 62, 4, 3, 8000),
(63, 63, 7, 2, 1800),
(64, 64, 6, 2, 1500),
(65, 65, 5, 1, 2500),
(66, 66, 9, 3, 1200),
(67, 67, 8, 1, 3500),
(68, 68, 10, 2, 7000),
(69, 69, 2, 2, 22000),
(70, 70, 1, 3, 15000),
(71, 71, 3, 1, 12000),
(72, 72, 4, 1, 8000),
```

```
(73, 73, 5, 2, 2500),
(74, 74, 7, 2, 1800),
(75, 75, 6, 3, 1500);
select*from Customers;
select*from Suppliers;
select*from Shippers;
select*from Payment_Methods;
select*from Products;
select*from Orders;
select*from Order_Items;
#a) Joins, Group By, Order By, Aggregations:

#1. Find the total revenue collected by each shipper.
#join shipper to orders to order_item . revenue = quan*price_each.group by shipper name
select sh.shipper_name,SUM(oi.quantity * oi.price_each) AS total_revenue from Shippers sh
join Orders o on sh.shipper_id = o.shipper_id join Order_Items oi on o.order_id = oi.order_id
group by sh.shipper_name order by total_revenue desc;

#2. Show the top #5 highest-spending customers along with their total payments.
#join customers to orders to order_items . sort by total spendings
select c.name, sum(oi.quantity * oi.price_each) as total_spent from customers c join orders o on c.customer_id=o.customer_id
join order_items oi on o.order_id=oi.order_id group by c.customer_id,c.name order by total_spent desc limit 5;

#3. Find product categories where the average selling price is greater than #8000.
#avg() gives average. having filters group
select category, avg(price) as avg_price from products group by category having avg(price)>8000;

#4. Show the total number of orders placed per city, sorted by highest to lowest.
# join customers with orders to get the city and use count() order per city
select city, count(o.order_id) as total_orders from customers c join orders o on c.customer_id=o.customer_id
 group by city order by total_orders desc;

#5. Find suppliers who supply more than #1 product category.
#count() and use having
select s.supplier_name, count(p.category) as new_category from suppliers s join products p on s.supplier_id = p.supplier_id
group by s.supplier_name having count(distinct p.category)>1;

#6. Show each order along with the number of items it contains.
# count() how many items .
select order_id, count(*) as num_of_items from order_items group by order_id order by order_id;

#b) Subqueries – Nested & Correlated:
#7. Find customers who have spent more than the average spending of all customers.
#this is subquery which i have use it after
```

```sql
select avg(customer_total) from(select sum(oi.quantity * oi.price_each) as customer_total from
Orders o join Order_Items oi on o.order_id = oi.order_id group by o.customer_id) as avg_table;
#main query
select c.name,sum(oi.quantity*oi.price_each) as total_spent from customers c join orders o
on c.customer_id=o.customer_id join order_items oi on o.order_id=oi.order_id group by
c.customer_id,c.name having sum(oi.quantity*oi.price_each)>(select avg(customer_total) from
(select sum(oi2.quantity*oi2.price_each) as customer_total from orders o2
join order_items oi2 on o2.order_id=oi2.order_id group by o2.customer_id) as avg_table);

#8. List all products whose price is higher than the average product price in their category.
# for each product inner query calculate
select avg(price) from products;
select p.* from products p where p.price>(select avg(price) from products where category=p.category);
#9. Show customers who placed at least one order with total_amount greater than #50,000.
select c.name from customers c join orders o on c.customer_id=o.customer_id join order_items oi on o.order_id=oi.order_id
group by c.customer_id,c.name having sum(oi.quantity*oi.price_each)>50000;

#10. Find customers who placed more orders than the average number of orders per customer.
#inner subquery finds order and avg() that and outer query keeps above
select c.name,count(o.order_id) as order_count from customers c join orders o on c.customer_id=o.customer_id
group by c.customer_id,c.name having count(o.order_id)>(select avg(order_count) from (select count(order_id)
as order_count from orders group by customer_id) as avg_orders);

#11. Find the most expensive product(s) in the catalog.
#campare each product price to highest
select max(price) from products;
select * from products where price=(select max(price) from products);

#c) Window Functions:
#12. Rank customers based on their total spending.
#rank() is a window function over() order by tell how to rank
 select c.name,sum(oi.quantity*oi.price_each) as total_spent,rank() over(order by sum(oi.quantity*oi.price_each)
 desc) as spending_rank from customers c join orders o on c.customer_id=o.customer_id join order_items oi
 on o.order_id=oi.order_id group by c.customer_id,c.name;

#13. Find cumulative sales amount by order date.
select o.order_date,sum(oi.quantity*oi.price_each) as daily_revenue,sum(sum(oi.quantity*oi.price_each)) over(order by o.order_date)
as cumulative_revenue from orders o join order_items oi on o.order_id=oi.order_id
group by o.order_date;
```

```sql

#14. Get each customer's order count and show their percentage contribution.
select c.name,count(o.order_id) as order_count,round(count(o.order_id)*100.0/(select count(*) from orders),2) as pct_contribution from
customers c join orders o on c.customer_id=o.customer_id group by c.customer_id,c.name order by order_count desc;

#15. Show the most recent order per customer.
#max per customer gives the latest date.simple and clean
select c.name,max(o.order_date) as most_recent_order from customers c join orders o on c.customer_id=o.customer_id group by c.customer_id,c.name order by
most_recent_order desc;

#16. List each product with its sales quantity and rank products within each category by total sales.
#partition by category restart rank for each category separetly
select p.product_name,p.category,sum(oi.quantity) as total_qty,rank() over(partition by p.category order by sum(oi.quantity) desc) as rank_in_category from products p
join order_items oi on p.product_id=oi.product_id group by p.product_id,p.product_name,p.category;
#d) CTE & Case:
#17. Categorize products as 'High', 'Medium', or 'Low' price using CASE.
#case is like if else when is condition and then is result and else is default vlaue
select product_name,price,case when price>50000 then 'high' when price between 5000 and 50000 then 'medium'
else 'low' end as price_label from products;

#18. Use a CTE to find top #3 customers by spending.
with customer_spending as (select c.name,sum(oi.quantity*oi.price_each) as total_spent from customers c join
 orders o on c.customer_id=o.customer_id join order_items oi on o.order_id=oi.order_id
group by c.customer_id,c.name) select name,total_spent from customer_spending order by total_spent desc limit 3;

#19. Use a CTE with CASE to classify customers by loyalty (based on number of orders).
with order_counts as (select c.name,count(o.order_id) as total_orders from customers c join orders o on
c.customer_id=o.customer_id group by c.customer_id,c.name) select name,total_orders,case when total_orders>=7 then 'loyal' when total_orders
 between 4 and 6 then 'regular' else 'new' end as loyalty_status from order_counts order by total_orders desc;

#20. Find monthly revenue growth percentage compared to the previous month.
with monthly_revenue as (select date_format(o.order_date,'%y-%m') as month,sum(oi.quantity*oi.price_each) as revenue
from orders o join order_items oi on o.order_id=oi.order_id group by month) select month,revenue,lag(revenue) over(order by month) as prev_revenue,
round((revenue-lag(revenue) over(order by month))/lag(revenue) over(order by month)*100,2) as growth_pct from monthly_revenue;
#21. Find top #2 customers per city based on total spending.
with city_spending as (select c.city,c.name,sum(oi.quantity*oi.price_each) as total_spent,rank() over(partition by c.city
order by sum(oi.quantity*oi.price_each) desc) as city_rank from customers c join
orders o on c.customer_id=o.customer_id join order_items oi on o.order_id=oi.order_id
```

```sql
group by c.city,c.customer_id,c.name) select city,name,total_spent,city_rank from city_spending where city_rank<=2;

#e) Miscellaneous Advanced Joins & Aggregations
#22. Find top #3 cities with highest sales revenue including shipper name.
# join 4 tables and order bu decs and limit 3
select c.city,sh.shipper_name,sum(oi.quantity*oi.price_each) as total_revenue from customers c join orders o
on c.customer_id=o.customer_id join order_items oi on o.order_id=oi.order_id join shippers sh
on o.shipper_id=sh.shipper_id group by c.city,sh.shipper_name order by total_revenue desc limit 3;

#23. List all orders with customer name, product name, supplier, and shipper.
select o.order_id,c.name,p.product_name,s.supplier_name,sh.shipper_name from orders o
join customers c on c.customer_id=o.customer_id join order_items oi on o.order_id=oi.order_id
join products p on oi.product_id=p.product_id join suppliers s on p.supplier_id=s.supplier_id
join shippers sh on o.shipper_id=sh.shipper_id;

#24. Show total sales per supplier along with average order value.
select s.supplier_name,sum(oi.quantity*oi.price_each) as total_sales,round(avg(oi.quantity*oi.price_each),2)
as avg_order_value from suppliers s join products p on s.supplier_id=p.supplier_id join
order_items oi on p.product_id=oi.product_id group by s.supplier_id,s.supplier_name order by total_sales desc;

#25. Show product categories that contributed more than #30% of total sales revenue.
with category_revenue as (select p.category,sum(oi.quantity*oi.price_each) as cat_revenue from products p
join order_items oi on p.product_id=oi.product_id group by p.category),grand_total as (select sum(cat_revenue)
as total from category_revenue) select cr.category,cr.cat_revenue,round(cr.cat_revenue/gt.total*100,2)
as revenue_pct from category_revenue cr,grand_total gt where cr.cat_revenue/gt.total*100>30;

# sir i took a help in 18,19,20,21,25  but i do understand the topics now and I'm learning these topics again so i dont forget
```